Touchless Computer Control System: Build Plan

A hand-gesture and voice control system for desktop computers. Built in two stages: a working **Minor** (gesture mouse) and a much larger **Major** (full touchless control suite). Hand this document to Antigravity (or any coding agent) and build phase by phase.

1. Goal and Scope

Minor project (working demo, about 2 weeks): webcam tracks the hand; index finger moves the cursor; pinch gestures click, right-click, scroll and drag.

Major project (about 8 weeks, built on the same codebase):

- Custom gesture training (user records own gestures, a small ML model learns them)
- Voice commands combined with gestures
- Presentation mode
- Gaming mode
- Accessibility settings panel
- Installable desktop app

Platform: Windows 10/11 first. macOS/Linux are stretch goals (cursor control permissions differ).

2. Tech Stack (all free for commercial use)

Purpose	Tool	License
Language	Python 3.10 to 3.12	PSF
Hand tracking	MediaPipe (Tasks API, HandLandmarker)	Apache 2.0
Camera and drawing	OpenCV (opencv-python)	Apache 2.0
Mouse and keyboard control	PyAutoGUI or pynput	BSD / LGPL
Custom gesture model	scikit-learn, NumPy, joblib	BSD
Offline voice commands	Vosk (small English model)	Apache 2.0
Better voice (optional)	faster-whisper / Whisper	MIT
Settings UI	CustomTkinter	MIT
Config	JSON files	n/a
Packaging	PyInstaller	GPL with bootloader exception, fine for distributing apps
Testing	pytest	MIT

Avoid: PyQt (GPL or paid), Google Web Speech API (rate-limited, not for commercial use), any code copied from GitHub repos without checking their license.

Reference repos (read for ideas, do not copy code unless the license allows):

- google-ai-edge/mediapipe: official hand landmark and gesture recognizer examples (Apache 2.0)
- asweigart/pyautogui: cursor and keyboard automation docs
- alphacep/vosk-api: offline speech recognition examples

There are many hobby "AI Virtual Mouse" repos on GitHub. Their licenses vary or are missing, so use them only as inspiration and write original code.

3. Repository Structure

```
touchless-control/
  README.md
  requirements.txt
  pyproject.toml
  config/
    default_config.json
    gestures.json            # gesture -> action mapping
    profiles/                # presentation.json, gaming.json, accessibility.json
  models/
    hand_landmarker.task     # downloaded from MediaPipe
    vosk-model-small-en-us/  # downloaded separately
    custom_gestures.joblib   # trained by the user in the app
  src/
    main.py                  # entry point
    core/
      camera.py              # webcam capture thread
      hand_tracker.py        # MediaPipe wrapper, returns landmarks
      smoothing.py           # One Euro filter / exponential smoothing
      gesture_engine.py      # rule-based gestures (minor)
      ml_gesture.py          # custom classifier (major)
      actions.py             # mouse, keyboard, scroll actions
      mode_manager.py        # switches Normal / Presentation / Gaming
      voice.py               # Vosk command listener
      config.py              # load/save settings
    ui/
      app.py                 # CustomTkinter main window
      settings_panel.py      # accessibility and sensitivity controls
      gesture_trainer.py     # record and train custom gestures
      overlay.py             # on-screen indicator and laser pointer
    utils/
      geometry.py            # distances, angles, normalization
      logger.py
  tests/
    test_geometry.py
    test_smoothing.py
    test_gesture_engine.py
  scripts/
    collect_data.py
    train_model.py
  docs/
    architecture.md
    user_guide.md
```

4. Architecture

1. **Camera thread** reads frames at 30 FPS and pushes them to a queue (keeps UI responsive).
2. **Hand tracker** runs MediaPipe and returns 21 landmarks per hand, normalized 0 to 1.
3. **Gesture engine** converts landmarks into a gesture name (rule-based in Minor, ML classifier in Major).
4. **Mode manager** looks up the gesture in the active profile and picks an action.
5. **Action layer** performs the mouse, keyboard or scroll event.
6. **Voice thread** (Major) listens for commands in parallel and sends them to the same action layer.
7. **UI** shows the camera preview, status, settings, and trainer. It talks to the core via a shared state object and event queue.

Keep core logic separate from UI so it can be tested without a camera.

5. Minor Project: Phase Plan

Phase 1: Setup and camera (day 1 to 2)

- Create the repo, virtual environment, requirements.txt.
- Webcam preview window with OpenCV, press Q to quit.
- **Done when:** a mirrored live preview runs at 25+ FPS.

Phase 2: Hand tracking (day 3 to 4)

- Integrate MediaPipe HandLandmarker (Tasks API, VIDEO or LIVE_STREAM mode).
- Draw the 21 landmarks on the preview.
- **Done when:** landmarks follow one hand reliably in normal indoor light.

Phase 3: Cursor movement (day 5 to 7)

- Use the index fingertip (landmark 8) as the pointer.
- Define an **active region** (a rectangle in the camera frame, about 60 to 70 percent of the frame) and map it to full screen resolution, so users never need to reach the frame edges.
- Add smoothing (exponential moving average first, then One Euro filter) to remove jitter.
- **Done when:** the cursor moves smoothly with no visible shaking while the hand is still.

Phase 4: Click and scroll gestures (day 8 to 11)

Rule-based gestures using normalized distances (divide by hand size, such as wrist to middle knuckle, so it works at any distance from the camera):

Gesture	Action
Index finger up only	Move cursor
Thumb and index pinch	Left click (hold = drag)
Thumb and middle pinch	Right click
Index and middle up together	Scroll (hand up/down)
Open palm	Pause tracking (safety)
Fist	Idle

- Add a click cooldown and hysteresis (different pinch-on and pinch-off thresholds) to avoid accidental double clicks.
- **Done when:** user can open a browser, click links, scroll a page and drag a window using only gestures.

Phase 5: Polish and demo (day 12 to 14)

- On-screen status text (current gesture, FPS).
- Config file for sensitivity, smoothing, thresholds.
- Failsafe: a hotkey (for example F8) to pause or quit; PyAutoGUI failsafe left on.
- README with install steps, a short demo video, and test notes.
- **Done when:** a 3-minute demo works without crashing.

6. Major Project: Phase Plan

Phase 6: Refactor and settings foundation (week 3)

- Move hard-coded thresholds into config.
- Add the CustomTkinter main window: start/stop, camera preview, status.
- Add the mode manager skeleton (Normal mode only).

Phase 7: Custom gesture training (week 3 to 4)

- **Data collection screen:** user names a gesture, holds it, app captures about 200 to 300 samples (21 landmarks x 3 coords = 63 features).
- **Normalization:** subtract the wrist position, divide by hand size, so the model ignores position and distance. Optionally mirror samples for left-hand support.

- **Training:** scikit-learn (start with RandomForest or an MLP), 80/20 split, show accuracy and a confusion matrix, save with joblib.
- **Prediction:** require a confidence threshold (for example 0.85) and stable results across 5 consecutive frames before triggering.
- **Mapping screen:** user assigns each gesture to an action (key press, mouse action, open app, run shortcut). Stored in gestures.json.
- **Done when:** user can train a new gesture in under 2 minutes and trigger an action with it.

Phase 8: Voice commands (week 5)

- Vosk small model, running in its own thread with a restricted grammar (a fixed command list gives much better accuracy than free dictation).
- Example commands: click, double click, right click, scroll up, scroll down, volume up, volume down, next slide, previous slide, start presentation, pause tracking, resume tracking, switch to gaming mode.
- Optional push-to-talk gesture or hotkey to avoid false triggers.
- Combined use: gesture to point, voice to confirm (for example point at an icon and say "open").
- **Done when:** 90 percent of spoken test commands in a quiet room trigger the right action.

Phase 9: Presentation mode (week 6)

- Swipe right/left = next/previous slide (send arrow keys).
- Pointing = on-screen laser pointer overlay (transparent topmost window with a red dot).
- Open palm hold = black screen (B key); fist = end show (Esc).
- Optional: zoom and annotate with a pen gesture.
- **Done when:** a PowerPoint or PDF presentation can be run end-to-end touch-free.

Phase 10: Gaming mode (week 6 to 7)

- Profile system: each profile maps gestures or hand position zones to keys (WASD, arrows, space, mouse buttons).
- Example profile: hand left/right/up/down from the center = A/D/W/S; pinch = jump; fist = attack.
- Lower latency settings (smaller frame resolution, higher FPS cap, lighter smoothing).
- Included demo profiles for a simple browser game and a racing game.
- **Note:** some online games block simulated input; test with offline or browser games.

Phase 11: Accessibility settings panel (week 7)

- Sensitivity, smoothing, and active region size sliders.
- **Dwell clicking:** hover for N seconds to click, for users who cannot pinch.
- One-hand or either-hand selection, left-handed mode.
- Larger click thresholds for users with tremors.
- High-contrast UI and large text options.
- Adjustable gesture hold time and cooldowns.
- Voice-only mode and gesture-only mode toggles.
- Save/load profiles per user.

Phase 12: Packaging and release (week 8)

- Error handling: no camera found, camera in use, model files missing.

- Logging to a file.
- PyInstaller build to a single Windows executable plus a setup guide.
- User guide with screenshots, final demo video, project report.

7. Key Technical Details (for the coding agent)

- Use `mediapipe.tasks.python.vision.HandLandmarker` with `hand_landmarker.task` (the older `mp.solutions.hands` is legacy). Pin the mediapipe version in requirements.txt.
- Mirror the frame horizontally (`cv2.flip(frame, 1)`) so movement feels natural.
- Screen mapping: `screen_x = interp(x, [region_left, region_right], [0, screen_w])` (same for y), then clamp.
- Smoothing: start with `smoothed = prev + (new - prev) / smoothing_factor`; upgrade to a One Euro filter for less lag at fast movement and less jitter at rest.
- Use `pyautogui.PAUSE = 0` and `moveTo(..., _pause=False)` for responsiveness; keep `FAILSAFE = True` (moving the cursor to a screen corner aborts).
- Run the camera and inference in a background thread; never block the UI thread.
- Normalize landmarks for ML: translate so the wrist is the origin, scale by the wrist-to-middle-MCP distance, then flatten to 63 features.
- Gesture stability: require the same prediction for N consecutive frames before firing an action.
- Keep all thresholds and mappings in JSON so the settings panel can edit them without code changes.

8. Testing Checklist

- Different lighting (bright, dim, backlit)
- Different distances from the camera (40 cm to 1.2 m)
- Left and right hand
- Different skin tones and hand sizes (test with several people if possible)
- Webcam resolutions: 640x480 and 1280x720
- Long runs of 30+ minutes (memory leaks, CPU use)
- Accidental trigger rate when the user is just talking or typing
- Unit tests for geometry, smoothing and the gesture engine using recorded landmark samples (no camera needed)

9. Risks and Fixes

Risk	Fix
Jittery cursor	One Euro filter, active region, larger dead zone
Accidental clicks	Hysteresis, cooldown, consecutive-frame check
Poor lighting	Brightness hint in UI, lower detection confidence threshold
High CPU use	Lower resolution, process every 2nd frame, cap FPS
Voice false triggers	Restricted grammar, push-to-talk option
Gaming input blocked	Document limits, support offline and browser games
Webcam privacy concerns	Process everything locally, never record or upload, show a clear camera-on indicator

10. Licensing and Monetisation Notes

- All listed core tools allow commercial use. Verify each license on its official page before selling, and keep a THIRD_PARTY_LICENSES file in the repo.
- MediaPipe model files (hand_landmarker.task) are provided by Google under Apache 2.0 terms; check the current model card.
- Vosk models vary in license (the small English model is Apache 2.0); check the specific model you download.
- Write your own code rather than copying from random repositories.
- Possible future monetisation: free core with paid Pro features (custom gesture limit, profiles, voice mode), a one-time license for accessibility users or institutions, or a B2B version for classrooms and presentations.

11. Future Upgrades (beyond the major project)

- Two-hand gestures (zoom, rotate)
- Head or eye tracking as an alternative input
- macOS and Linux support
- Cloud-synced profiles (opt-in)
- Gesture packs for specific apps (Photoshop, video editors, music players)
- Mobile or Raspberry Pi version for smart-TV control

12. Prompts to Give Antigravity

Use one prompt per phase and review before moving on.

Kickoff prompt:

> Read PLAN.md in full. Create the repository structure from section 3, a requirements.txt with pinned versions, and a README. Do not write feature code yet. Show me the folder tree and wait for approval.

Phase prompts (copy, replace the phase number):

> Implement Phase N from PLAN.md exactly as specified. Keep core logic separate from UI, put thresholds in config, add unit tests where a camera is not needed, and run the tests. Tell me how to run and manually verify the "Done when" criteria. Do not start the next phase.

Debug prompt:

> The cursor is jittery / clicks fire twice / FPS is low. Profile the pipeline, identify the cause, and fix it using the techniques in section 7 and section 9 of PLAN.md. Explain the change in two sentences.

Review prompt (before the viva or demo):

> Review the whole codebase for bugs, unhandled errors, hard-coded values, and license issues. Produce a short list of fixes ordered by risk.

13. Suggested Timeline

Week	Work
1	Phases 1 to 3 (camera, tracking, cursor movement)
2	Phases 4 to 5 (clicks, scroll, polish, **minor submission**)
3	Phase 6 and start of Phase 7 (UI, trainer)
4	Finish Phase 7 (custom gestures)
5	Phase 8 (voice)

Week	Work
6	Phase 9 and start of Phase 10 (presentation, gaming)
7	Finish Phase 10, Phase 11 (accessibility)
8	Phase 12 (packaging, report, demo video, **major submission**)